

Benefits for the Findigs Data Team

v3 · Prepared by ContractGate · May 2026 · Confidential

<15ms

p99 validation latency

100%

events audited

Bi~~di~~ir

ingest + egress
enforcement

Zero

PII leakage to clients

Prom

Prometheus + Grafana

Executive Summary

Findigs processes high-velocity, multi-source applicant data — payroll, bank feeds, identity documents, background reports — and translates it into real-time leasing decisions. The primary intake layer is three Property Management System (PMS) integrations — Yardi, AppFolio, MRI — whose exports are notoriously inconsistent. Equally critical is the *outbound* path: `feed_nomad_applications` and `feed_bridge_applications` expose derived values like `verified_income` to external clients, where any drift or PII leak is a customer-visible incident. ContractGate now enforces a semantic contract at **both** boundaries — ingest *and* egress — logs every event for FCRA auditability, quarantines violations before they reach screening decisions, and ships full Prometheus observability for the validation engine.

WHAT'S NEW IN v3

Since the v2 briefing, ContractGate shipped: **egress validation** (symmetric outbound contract enforcement), **egress PII & leakage guard** (mask outbound + reject undeclared fields), **contract publication & subscription** (share contracts by stable ref; subscribers see drift), **provider-consumer collaboration** (cross-org comments + proposal review), **Prometheus observability v1** (six metrics + Grafana dashboard), **native date type** (calendar-aware date validation, not just regex), and **Kafka / Kinesis ingress consumers**. Each is detailed below.

01 · Data Quality & Semantic Validation — Taming PMS Data

All 34 PMS API source tables — across Yardi, AppFolio, MRI — currently carry zero source-level data tests. There are no `not_null`, uniqueness, or `accepted_values` checks on any raw ingestion table. Every field assumption embedded in the staging models is unverified at the boundary. ContractGate replaces this gap with a single version-controlled semantic contract per vendor, expressing not just shape (types, required fields) but meaning (allowed values, business glossary, derived metrics).

- **One contract per PMS vendor.** Define a `yardi_lease.yaml`, `appfolio_lease.yaml`, `mri_lease.yaml` — each capturing the exact quirks of that vendor's output. No more per-service workaround code scattered across staging models.

- **Catch vendor drift at the gate.** When a PMS provider silently renames a field or adds an unhandled code value, ContractGate rejects the event at the gateway — not inside your scoring model hours later.
- **Quarantine, don't discard.** Violating events are held for inspection and replay once the upstream issue is fixed. No data is silently dropped or zeroed out.
- **Computed metrics as contract fields.** Derived values like income-to-rent ratio are defined inside the contract — every service uses the same formula, always.
- **Native date type.** NEW Calendar-aware date validation. `type: date` rejects `2026-15-17` (month 15) at the gate, where the v2 regex-only approach would have let it through. ODCS-compatible (`logicalType: date`); existing `type: string` + pattern contracts continue to work unchanged.

LIVE FINDING · RENT FIELD PROLIFERATION

"Monthly rent" is represented differently across all three vendors with no shared definition or validation. Yardi exposes `monthlyrentamount` (commented out in staging: *"Use later once we figure out rents for other PMSs"*), plus `unit_market_rent` and `unit_effective_rent_min/max`. AppFolio stores `rent`, `marketrent`, and `mostrecentrent` as raw strings, coerced inline via `replace(col, ',', ' '::float` with no bounds check. MRI exposes five distinct rent concepts: `unit_base_rent`, `unit_lro_rent_min`, `unit_effective_rent_min`, `unit_effective_rent_max`, and `unit_potential_rent`. A single ContractGate contract would enforce a canonical `monthly_rent` field — numeric, non-null for active leases, greater than zero — before any of this reaches the warehouse.

LIVE FINDING · BRITTLE LEASE KEY CONSTRUCTION

Each vendor constructs `pms_lease_key` via completely different logic — Yardi uses `yardi_resident_id` directly (the staging model notes: *"Crazy as it sounds, yardi_resident_id is unique over the leases table"*), AppFolio hashes two columns, and MRI hashes five. None of the component columns have `not_null` tests. A null in any upstream component silently produces a garbage hash key that propagates downstream undetected. ContractGate enforces `not_null` on each key component at the source boundary.

Example: Canonical rent contract enforced across all three vendors

```
version: "1.0"
name: "pms_lease_rent"
description: "Canonical rent contract – normalises across Yardi, AppFolio, and MRI"
ontology:
  entities:
    - name: pms_source
      type: string
      required: true
      enum: [yardi, appfolio, mri]
    - name: monthly_rent
      type: number
      required: true
      min: 0.01
      # Yardi: monthlyrentamount; AppFolio: rent (string → float); MRI: unit_base_rent
    - name: lease_start_date
```

```

    type: date          # NEW — calendar-aware, not just regex
    required: true
  - name: pms_lease_key
    type: string
    required: true
    pattern: "^[a-f0-9]{32}$|^[0-9]+$"
metrics:
  - name: income_to_rent_ratio
    formula: "verified_income / monthly_rent"
    description: "Primary RTI signal; same verified_income path across all vendors"

```

02 · Sub-15ms Validation — No Latency Tax

Findigs promises real-time screening decisions to property managers. Any middleware that adds meaningful latency to the critical path is a non-starter. ContractGate's Rust-based validation engine is purpose-built for this constraint.

- **p99 < 15ms.** End-to-end contract validation — ontology checks, glossary lookups — fits inside a synchronous request budget.
- **In-process or sidecar.** Deploy as an Axum-based sidecar beside each ingestion service. No round-trip to a remote validator.
- **Zero allocations on the hot path.** Pre-compiled contract structures; per-event overhead is a single payload pass.
- **Scales with event volume.** Stateless validation — add instances as throughput grows.
- **Prometheus + Grafana out of the box.** **NEW** `/metrics` endpoint exposes request rate, validation p50/p95/p99 histograms, violation rate by kind, quarantine rate, and active contract count. Ships with a six-panel Grafana dashboard. Metric writes are post-response — the p99 budget is unaffected.

03 · Outbound Enforcement — Catching Leaks Before Clients See Them

NEW SECTION

The most consequential ContractGate addition since v2: **egress validation**. The same contract that gates ingestion now gates the response path. For Findigs this directly covers `feed_nomad_applications` and `feed_bridge_applications` — the feeds that expose `verified_income` and other derived screening signals to external clients.

- **Symmetric enforcement.** `POST /egress/{contract_id}` reuses the same `validate()` engine as ingest. One contract, one rule set, both directions. No drift between «what we accept» and «what we send.»
- **Three disposition modes.** `block` (default — reject the entire response on any violation), `fail` (return 422 with violations), or `tag` (annotate outcomes, ship the payload anyway). Pick per integration based on client tolerance.
- **Egress PII guard.** RFC-004 transform engine runs on the outbound payload before the response is built. Raw PII never appears in `feed_*` responses, regardless of what the upstream pipeline computed.

- **Leakage modes for undeclared fields.** `egress_leakage_mode` per contract version: `off` (compatibility), `strip` (silently drop undeclared fields), or `fail` (reject the response). Stops accidental field exposure cold.
- **Salt continuity.** A hashed `applicant_id` on egress matches the hash on ingest — downstream joins keep working without unmasking.
- **Audit log tags direction.** Every audit row now carries `direction = 'ingress'` or `'egress'`. Query «what did we send Nomad last Tuesday?» with one SQL filter.

LIVE FINDING · HARDCODED EVICTION LOGIC IS A COMPLIANCE RISK

MRI lease staging currently determines `is_eviction` and `is_skip` via hardcoded `pms_connection_id` integers and opaque reason codes: `case when pms_connection_id = 34 and vreason_upper in ('A1', 'A2') then true`. Any `pms_connection_id` not in the hardcoded list silently returns null — meaning eviction flags are silently suppressed for unrecognised connections. For a screening platform with FCRA obligations, an undetected null eviction flag is a material compliance risk. A ContractGate contract would define the accepted `vreason` values per connection and quarantine any unhandled code for human review before it reaches a screening decision. With **egress enforcement** the same contract guarantees no eviction-flag drift reaches downstream feeds.

04 · Audit Log & Regulatory Compliance

FCRA and FHA require screening decisions to be traceable to accurate, complete data. ContractGate's immutable audit log provides exactly this — a timestamped record of every event validated, the contract version that actually matched, and the outcome.

- **Immutable per-event records.** Every event references the exact contract version that validated it — prove, months later, what rules were in effect. The recorded version is the one that matched, never a default.
- **Direction-tagged.** NEW Audit rows now carry `direction` (ingress or egress), so you can prove both «what we accepted» and «what we sent» in the same query.
- **Quarantine trail.** Rejected events are preserved with the violation reason for post-incident review. Nothing is silently dropped.
- **Quarantine guard at deploy time.** ContractGate blocks a new contract version from going live if pending quarantine events exist for that feed — preventing a rule change from quietly masking existing violations.
- **Contract versioning.** Update screening rules without breaking decision traceability — each version is independently addressable and carries `deployed_by`, `deployed_at`, and `source` metadata.
- **Audit log queryable via SQL.** The `active_contracts_public` view is granted to authenticated users — query contract history directly from your Snowflake pipeline with standard SQL.

05 · Sharing & Cross-Org Collaboration NEW SECTION

PMS integrations evolve. A field gets renamed, a code list grows, a vendor adds a new lease status. Two new subsystems let Findigs collaborate with property managers (or internally across teams) on the canonical contract — without round-tripping through Slack screenshots and email threads.

Contract Publication & Subscription NEW

- **Publish a contract version by stable ref.** `POST /contracts/{id}/versions/{v}/publish` mints a 24-hex ref (or link-token for private sharing). Visibility: `public`, `link`, or `org`.
- **Snapshot or subscribe mode.** Consumers import via `POST /contracts/import-published` in `snapshot` mode (frozen copy) or `subscribe` mode (track the source). Subscribers see an "↑ Update available" badge when the publisher ships a new version, and a "source revoked" badge if the publisher pulls it.
- **Drift visibility.** The dashboard polls `import-status` for every subscribe-mode contract and surfaces the diff before you adopt it.

Provider–Consumer Collaboration NEW

- **Roles per contract.** Owner / editor / reviewer / viewer — grant a property manager `reviewer` on `yardi_lease` without exposing the rest of your contract catalog.
- **Field-anchored comments.** Threaded notes attached to a specific field; mark resolved when the question is settled. No more "see row 47 of the spreadsheet."
- **Proposal-based change review.** Editors propose a YAML change; reviewers approve or reject; owner applies. Status machine: `open` → `approved/rejected` → `applied`. The owner is in the loop on every version bump.

06 · Stream Ingress — Kafka & Kinesis NEW SECTION

Beyond the synchronous `POST /ingest` path, ContractGate now runs as a managed consumer against Kafka and Kinesis streams — validating events in flight without bolting validation onto every producer.

- **Native Kafka consumer.** Bind a contract to a topic; failing events route to a dead-letter topic with the violation reason. The Confluent connector ships separately for teams that prefer the Connect framework.
- **Native Kinesis consumer.** Same model for AWS-native streams; quarantine writes share the SQL audit table so observability stays unified.
- **Same engine, same metrics.** Stream consumers use the identical `validate()` path as the HTTP ingest handler. p99 latency and Prometheus instrumentation apply unchanged.

07 · Data Team Developer Experience

Data contracts are only as good as the team's ability to write, review, deploy, and iterate on them. ContractGate keeps the authoring surface minimal and familiar — and ships a full deploy toolchain so contracts go from draft to production in a single command.

- **Plain YAML — no DSL.** Contracts read like documentation. Non-engineers can review and approve them in pull requests.
- **Version-controlled by default.** Change management, diffs, and rollbacks are standard git operations.
- **Self-documenting glossary.** Each field carries a business description and constraint note — no more spreadsheet data dictionary.
- **Metrics as code.** Income-to-rent ratio, fraud thresholds — defined once in the contract, used consistently everywhere.
- **One-command deploy with full audit trail.** `cg deploy-contract yardi_lease.yaml --source git --deployed-by alex` atomically promotes a contract to stable, deprecates the prior version, and records who deployed it and when.
- **Dry-run before you commit.** `--dry-run` validates the contract and reports violations against live events without deploying.
- **Infer contracts from a CSV.** Upload a raw PMS export and ContractGate auto-drafts the contract — types, nullability, and field patterns inferred automatically.
- **Fork and refine.** Start from an existing contract and fork it to your namespace. No blank-page problem when adding a fourth PMS vendor.

LIVE FINDING · CROSS-VENDOR METRIC INCONSISTENCY

`is_term_mtm` (month-to-month lease indicator) is computed three different ways with no cross-vendor consistency test: Yardi uses `lease_term_in_months <= 1`, AppFolio uses a string match on `lower(trim(status)) = 'month to month'`, and MRI uses a code lookup on `lower(leasnm) in ('mtm', 'm2m')`. Similarly, `verified_income` — the primary input to the income-to-rent ratio — has two independent computation paths with a known divergence noted inline in the model. That value flows directly into `feed_nomad_applications` and `feed_bridge_applications`, which expose it to external clients. With **egress validation**, the divergence is now caught at the response boundary even if it slips past ingest — defining these metrics once in a contract eliminates the ambiguity entirely.

How ContractGate Compares

Capability	Ad-hoc Validation	Schema Registry	ContractGate
Semantic type contracts	✗	<i>Partial</i>	✓
Sub-15ms p99 enforcement	✗	✗	✓
Immutable audit log	✗	✗	✓

Quarantine + replay	X	X	✓
Human-readable YAML contracts	X	<i>Partial</i>	✓
FCRA-traceable decisions	X	X	✓
CLI deploy + dry-run	X	X	✓
CSV auto-inference	X	X	✓
Contract catalog + forking	X	X	✓
Symmetric egress enforcement NEW	X	X	✓
Egress PII + leakage guard NEW	X	X	✓
Calendar-aware <code>date</code> type NEW	X	<i>Partial</i>	✓
Publish + subscribe (drift alerts) NEW	X	X	✓
Cross-org comments + proposals NEW	X	X	✓
Prometheus + Grafana built-in NEW	X	<i>Partial</i>	✓
Kafka + Kinesis consumers NEW	X	<i>Partial</i>	✓

Why This Fits Findigs

Findigs' screening pipeline currently has no enforcement layer at the PMS source boundary — and no enforcement at all on the outbound client feeds. All 34 raw source tables across Yardi, AppFolio, and MRI carry zero data tests. Monthly rent has no canonical definition. Lease keys hash to garbage on nulls. Eviction flags silently return null for unrecognised connections. And `verified_income` — computed via two diverging paths — flows unchecked into `feed_nomad_applications` and `feed_bridge_applications`, where it is exposed to external clients with no symmetric validation on the way out. ContractGate v3 addresses each of these: one contract per vendor, enforced at *both* ingest *and* egress, version-controlled, with PII masking on the outbound path, every event queryable via SQL, every metric scraped by Prometheus, and a publish/subscribe model so the canonical contract can be shared with property managers without spec drift. The new egress and observability stacks shipped in May 2026 — Findigs can be in production on the Yardi feed within two weeks, with the Nomad and Bridge feed responses guarded from day one.